



Фаза 3.1: Изолированный L1/L2 Support Worker — Стерильная Working Memory и Сжатие Контекста

Авторы: Казбек & AI Engineering Lead (CAE Data LLC R&D)

Статус: Архитектурная спецификация реализации (Implementation Spec)

1. Проблема, которую решает Фаза 3.1

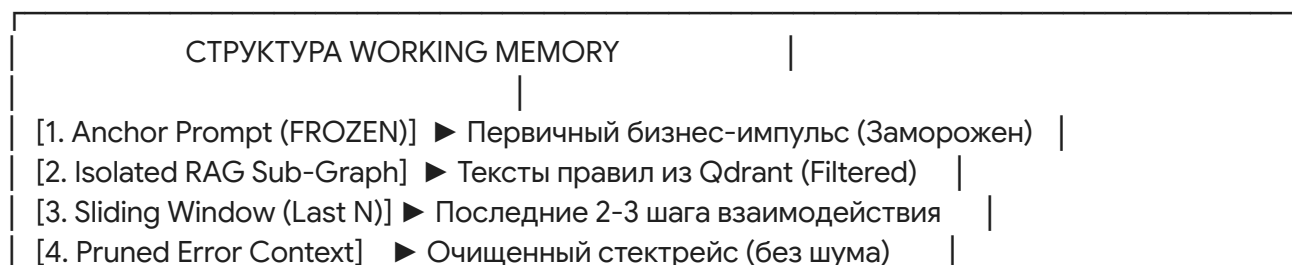
В классических фреймворках (LangChain / базовый LangGraph) контекст представляет собой монотонно растущий список сообщений `messages: list[BaseMessage]`.

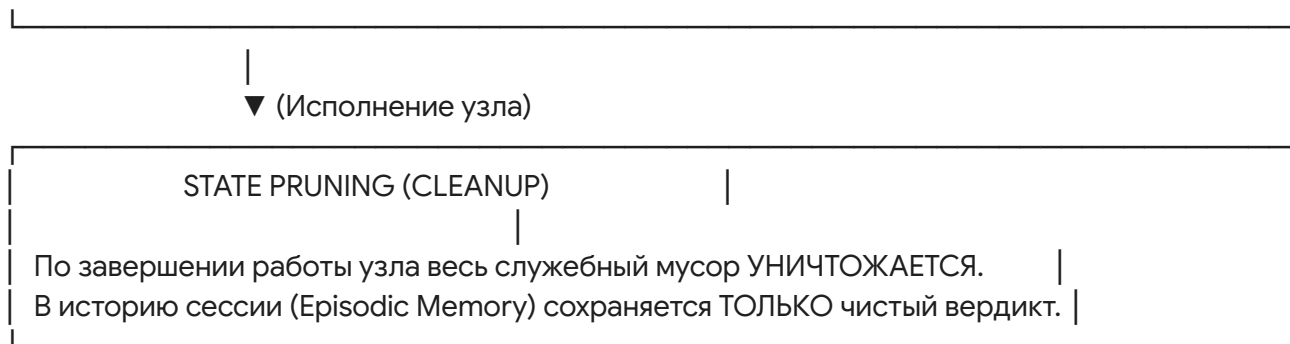
При N итерациях дебатов или отладки дефекта:

1. Длинные стектрейсы Python/Spark и дампы SQL засоряют «брюхо» контекста (20% — 80% токенов).
2. Теряется первоначальная системная цель («Anchor Prompt»).
3. Внимание модели размывается, растёт стоимость токенов, модель начинает повторять свои прошлые ошибочные выводы.

2. Архитектура Стерильной Working Memory

В Фазе 3.1 мы вводим жесткое правило: **Working Memory** — это не история диалога, а вычислительный буфер текущего шага.





3. Три механизма управления контекстом

A. Anchor + Sliding Window (Заморозка первоначального импульса)

Для предотвращения смыывания цели («Attention Smearing») системный контекст разделяется на две зоны:

- **Static Anchor Zone (Замороженный якорь):** Первичная бизнес-постановка задачи, правила CAE Data LLC и контракты ответа. Эта зона никогда не смещается и не сокращается.
- **Dynamic Sliding Window (Плавающее окно):** Хранит только последние K реплик (например, $K = 2$). Все предыдущие $N - K$ сообщений сворачиваются в резюме или вырезаются.

B. Semantic Pruning (Семантическая зачистка логов)

Перед передачей лога ошибки в LLM выполняется детерминированная фильтрация:

1. **Удаление повторов:** Повторяющиеся строки одинаковых ошибок компрессируются в формат [Repeated X times].
2. **Очистка системных фреймов:** Из Python Traceback вырезаются стандартные внутренние пути библиотеки (/usr/local/lib/python3.11/...), оставляется только код пользовательского приложения (mcp_worker/core/...).
3. **Сжатие данных:** Если в логе лежит дамп всей таблицы, он обрезается до 3 репрезентативных строк.

C. Hierarchical Summarization (Иерархическая дистилляция)

Если дебаты между Worker и Supervisor или круги рефлексии превышают $N = 3$ итераций:

1. Промежуточная переписка передается легкой модели-сжимателю.
2. Сжиматель возвращает структурированный контекст из 3 пунктов:
 - **Что уже попробовали:** (например, fill_на не сработал из-за типов).
 - **Что заведомо неверно:** (например, поле не принимает 0.0).
 - **Текущая гипотеза:** (требуется преобразование схемы).
3. Весь длинный лог прошлых 3 итераций заменяется этим компактным резюме.

4. Схема данных (Working Memory Pydantic Model)

Для управления Working Memory создается изолированная структура состояния, совместимая с LangGraph:

```
from pydantic import BaseModel, Field
from typing import List, Dict, Any, Optional

class WorkingMemoryState(BaseModel):
    # Anchor Zone
    anchor_prompt: str = Field(description="Замороженная системная цель и аксиомы")
    isolated_rag_rules: List[str] = Field(default_factory=list, description="Строго отфильтрованные правила Qdrant")

    # Pruned Error Payload
    pruned_error_log: str = Field(description="Очищенный от системного шума лог ошибки")

    # Dynamic Window
    sliding_history: List[Dict[str, str]] = Field(
        default_factory=list,
        description="Плавающее окно последних K сообщений (max 2-3)"
    )
```

```
# Hierarchical Summary
distilled_summary: Optional[str] = Field(default=None, description="Иерархическое резюме прошлых нереализованных гипотез")
```

5. Алгоритм работы функции `prune_and_compress_context()`

```
def prune_and_compress_context(
    raw_error: str,
    history: List[Dict[str, str]],
    max_window: int = 2
) -> WorkingMemoryState:
    """
    Принимает сырой контекст сессии и возвращает стерильную Working Memory.
    """

    # 1. Семантическая зачистка лога ошибки (Semantic Pruning)
    cleaned_error = remove_system_frames_from_traceback(raw_error)
    cleaned_error = truncate_data_dumps(cleaned_error)

    # 2. Ограничение окна истории (Sliding Window)
    recent_history = history[-max_window:] if len(history) > max_window else history

    # 3. Иерархическое сжатие при длинной истории
    distilled = None
    if len(history) > max_window:
        distilled = summarize_prior_attempts(history[:-max_window])

    return WorkingMemoryState(
        anchor_prompt="CAE Data LLC DataOps L1/L2 Support Agent Anchor Instructions...",
        pruned_error_log=cleaned_error,
```

```
sliding_history=recent_history,  
distilled_summary=distilled  
)
```

6. Результат применения в оркестраторе

Параметр	Без Working Memory (Старый подход)	С Working Memory & Context Compression
Размер контекста на 4-м круге	~12,000–18,000 токенов	~1,200–2,000 токенов (стабильно)
Риск зацикливания/деградаци и	Высокий (после 3 итерации)	Близкий к 0
Фокус на первичной цели	Теряется в "брюхе" контекста	100% (Благодаря Anchor Prompt)
Чистота истории (Episodic Memory)	Загрязнена трейсбэками	Содержит только финальные решения